

Visió per Computador

Pràctica 2: Detecció de contorns

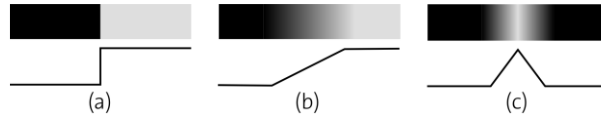
NEUS OLLER MATAS

Enginyeria Informàtica

Universitat Rovira i Virgili

Exercici 1: Identificació de models de contorns

A continuació teniu les 3 representacions ideals de models de contorns que poden aparèixer a les imatges: step edge (a), ramp edge (b) i roof edge (c).



Identifiquen aquests models en les imatges contingudes a la carpeta «imatges_contorns».

Contorn: es defineix com les línies que permeten traçar els límits d'una figura. Per definició una imatge de contorns és binària: cada píxel pertany o no al contorn.

La detecció de contorns sempre es fa en imatges de grisos (intensitat). En cas de tenir una imatge en color caldrà descartar la informació de color.

- *camisas_bn*: ramp edge
- *d95*: roof edge
- *d95a2*: step edge
- *frase_bn*: ramp edge

Exercici 2: Aplicant detectors de contorns

Els contorns es poden obtenir a partir del càlcul del gradient, específicament, calculant el mòdul del gradient:

$$|G| = \sqrt{G_x^2 + G_y^2} \approx |G_x| + |G_y|$$

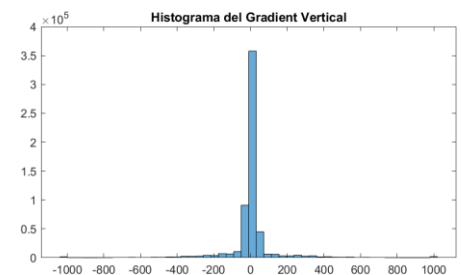
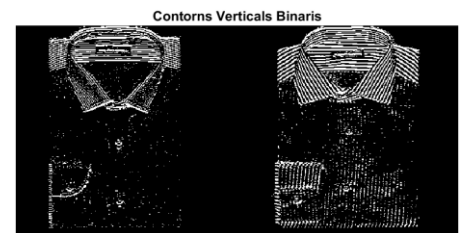
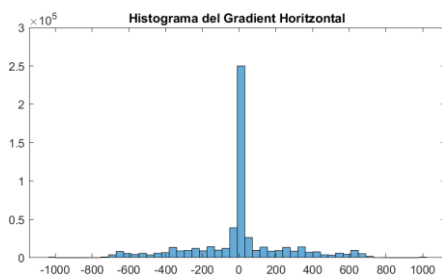
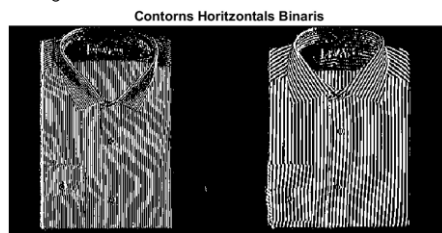
Sent G_x i G_y els gradients horitzontal i vertical respectivament. Per calcular el gradient es poden utilitzar diversos operadors (Roberts, Sobel, Prewitt) tots ells basats en el càlcul de la primera derivada.

Un cop calculat el gradient, per obtenir la imatge de contorns (binaris) cal aplicar thresholding (binarització).

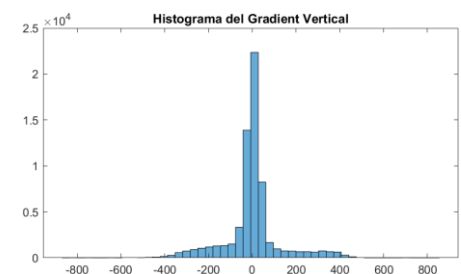
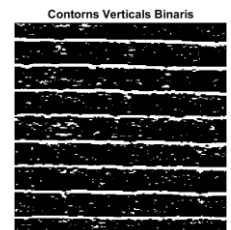
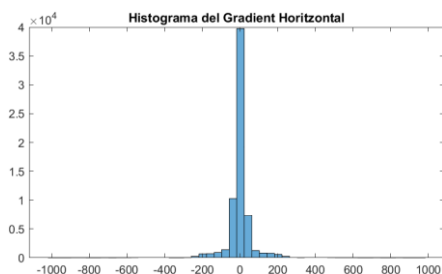
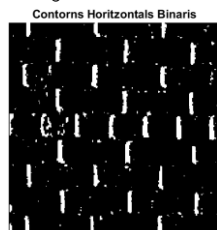
Per calcular G_x cal convolucionar la imatge amb l'operador gradient oportú, el mateix per G_y .

1. De les imatges que heu analitzat a l'exercici anterior obtingueu els contorns calculant la imatge gradient utilitzant l'aproximació que s'indica a la fórmula anterior. Feu servir l'operador Sobel.

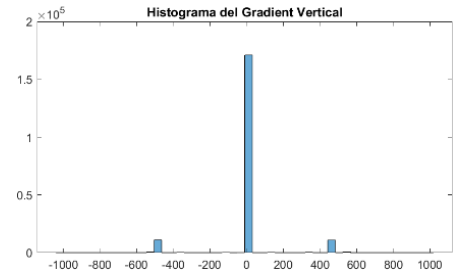
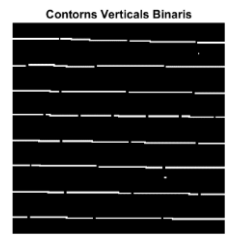
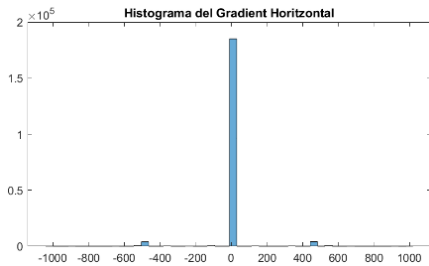
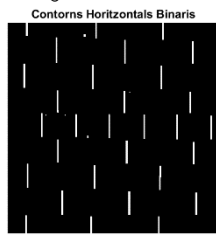
Imatge camisas_bn



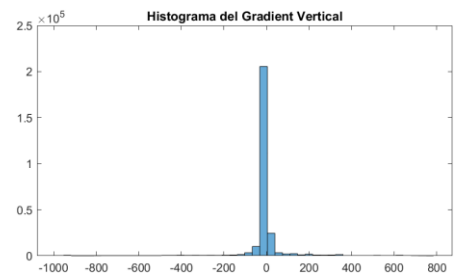
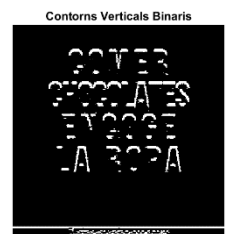
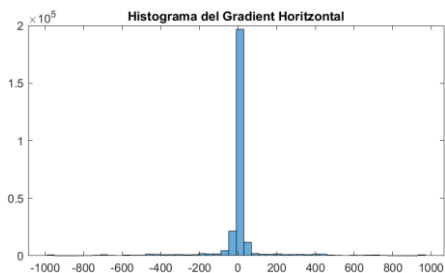
Imatge d95



Imatge d95a2



Imatge frase_bn



```
im = imread('frase_bn.bmp');

Gx = imfilter(double(im), fspecial('sobel'));
Gy = imfilter(double(im), fspecial('sobel'));

threshold_value = 50;

binary_gradient_x = Gx > threshold_value;
binary_gradient_y = Gy > threshold_value;

binary_gradient_x_uint8 = uint8(binary_gradient_x) * 255;
binary_gradient_y_uint8 = uint8(binary_gradient_y) * 255;

figure;
subplot(2, 2, 1);
imshow(binary_gradient_x_uint8);
title('Contorns Horitzontals Binaris');

subplot(2, 2, 2);
imshow(binary_gradient_y_uint8);
title('Contorns Verticals Binaris');

subplot(2, 2, 3);
histogram(Gx(:), 50);
title('Histograma del Gradient Horitzontal');

subplot(2, 2, 4);
histogram(Gy(:), 50);
title('Histograma del Gradient Vertical');
```

2. Trobeu la relació entre el model de contorn (a/b/c) que tenen les imatges i el resultat obtingut en la imatge de contorns (línies primes/gruixudes).

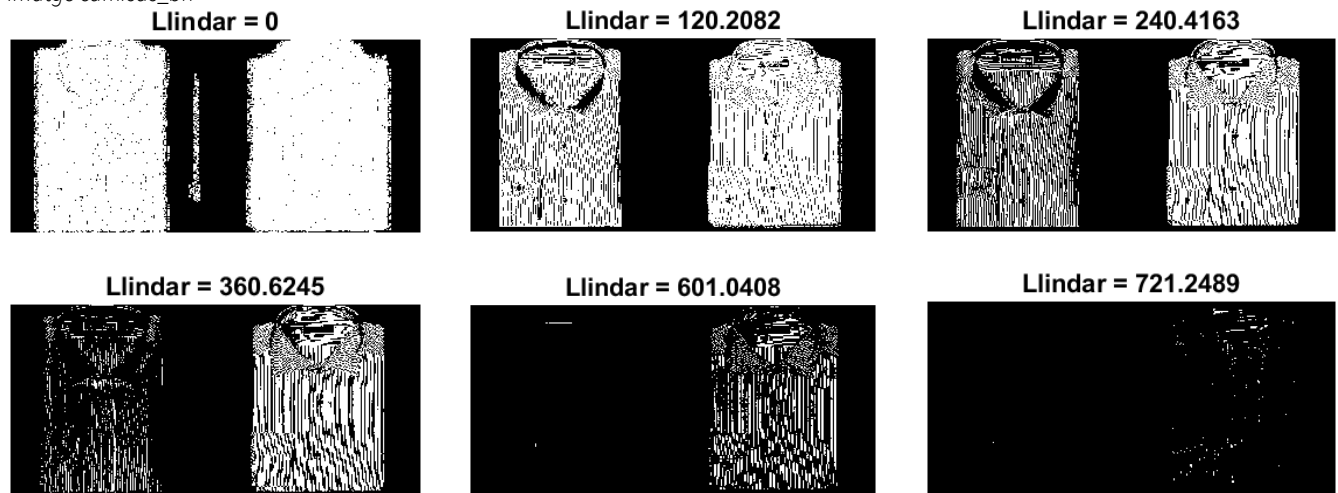
Step Edge: Aquest model de contorn té contorns nets. Les línies de contorn generades són primes ja que representen canvis bruscos en la intensitat de la imatge.

Ramp Edge: Els contorns generats per aquest model seran més gruixuts en comparació amb els contorns dels *step edges*. Les línies de contorn semblaran més esteses, estarà representant una transició gradual en la intensitat de la imatge.

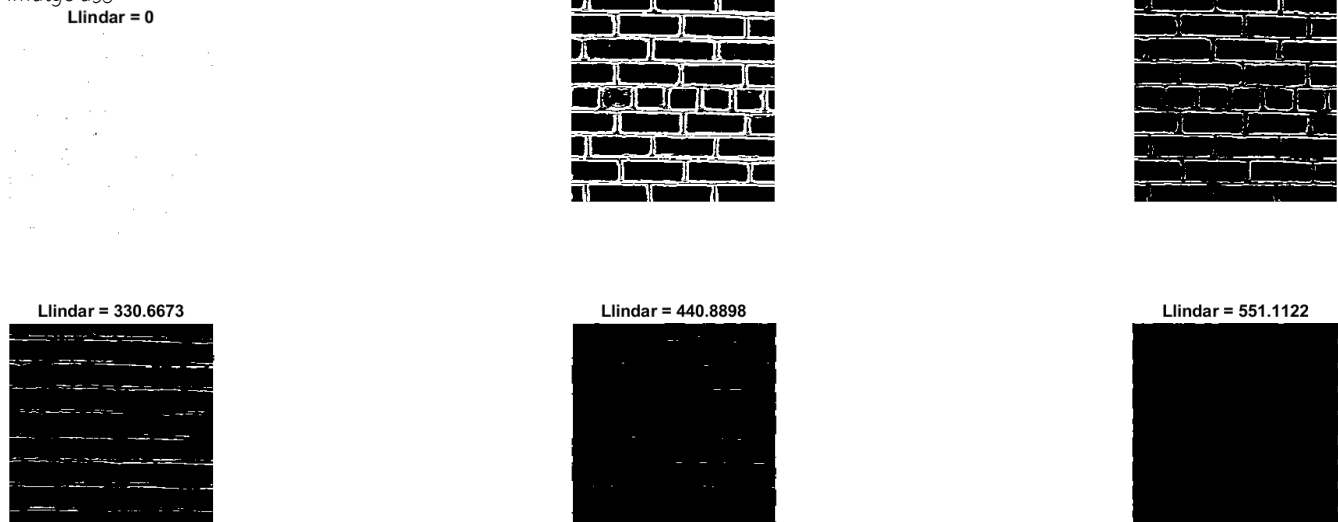
Roof Edge: Aquest model de contorn té una forma triangular en el perfil de la intensitat dels píxels. Les línies de contorn resultants podrien semblar més gruixudes i tindrien una forma triangular o de "V" en lloc de ser primes com les dels *step edges*.

3. En la fase de *thresholding* feu proves utilitzant llindars diferents i observeu com canvien els resultats depenent del llindar.

Imatge camisas_bn

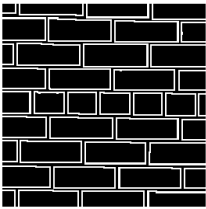


Imatge d95

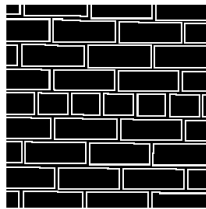


Imatge d95a2

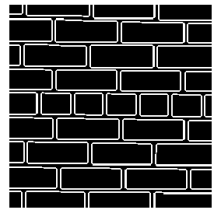
Llindar = 0



Llindar = 113.3333



Llindar = 453.3333



Llindar = 566.6667



Llindar = 793.3333



Llindar = 906.6667



Imatge frase_bn

Llindar = 0



Llindar = 112.1943



Llindar = 224.3886



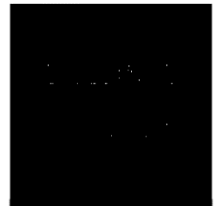
Llindar = 448.7771



Llindar = 673.1657



Llindar = 785.3599



```
im = imread('camisas_bn.bmp');
if ndims(im) == 3 && size(im, 3) == 3
    im_grisos = rgb2gray(im);
else
    im_grisos = im;
end

Gx = imfilter(double(im_grisos), fspecial('sobel'));
Gy = imfilter(double(im_grisos), fspecial('sobel'));
gradient = sqrt(Gx.^2 + Gy.^2);

valors_llindar = linspace(0, max(gradient(:)), 10);

figure;
for i = 1:length(valors_llindar)
    llindar_actual = valors_llindar(i);

    im_binaritzada = gradient > llindar_actual;

    subplot(3, 4, i);
    imshow(im_binaritzada);
    title(['Llindar = ', num2str(llindar_actual)]);
end
```

Exercici 3: Estudi de l'efecte de suavitzat sobre la imatge de contorns

1. Sobre la imatge d95 apliqueu un filtre de suavitzat (mitjana o gaussià) de mida 3x3 i després obteniu les línies de contorn utilitzant el mètode de l'exercici anterior. L'objectiu és obtenir en la imatge resultat totes les línies de contorn de la imatge original.

```
im = imread('d95.bmp');

im_suavitzada = imfilter(im, fspecial('average', [3 3]));

if ndims(im) == 3 && size(im, 3) == 3
    im_grisos = rgb2gray(im);
else
    im_grisos = im;
end

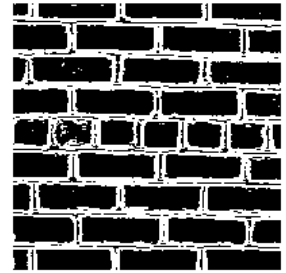
Gx = imfilter(double(im_grisos), fspecial('sobel'));
Gy = imfilter(double(im_grisos), fspecial('sobel'));
gradient = sqrt(Gx.^2 + Gy.^2);

llindar = mean(gradient(:));

im_binaritzada = gradient > llindar;

imshow(im_binaritzada);
title(['Llindar = ', num2str(llindar)]);
```

Llindar = 93.8974



2. Repetiu el mateix procediment anterior però aplicant filtres de suavitzat de mida major (7x7, 13x13). En la fase de thresholding apliqueu el mateix llindar que abans.

```
im = imread('d95.bmp');

im_grisos = im2gray(im);

mides_filtre = [7, 13];

Gx = imfilter(double(im_grisos), fspecial('sobel'));
Gy = imfilter(double(im_grisos), fspecial('sobel'));
gradient = sqrt(Gx.^2 + Gy.^2);
llindar = mean(gradient(:));

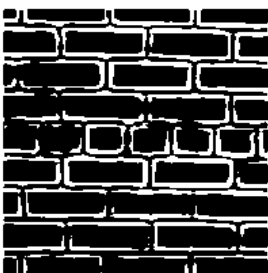
for i = 1:length(mides_filtre)
    im_suavitzada = imfilter(im_grisos, fspecial('average', [mides_filtre(i) mides_filtre(i)]));

    Gx = imfilter(double(im_suavitzada), fspecial('sobel'));
    Gy = imfilter(double(im_suavitzada), fspecial('sobel'));
    gradient = sqrt(Gx.^2 + Gy.^2);

    im_binaritzada = gradient > llindar;

    figure;
    imshow(im_binaritzada);
    title(['Filtre de suavitzat ', num2str(mides_filtre(i)), 'x', num2str(mides_filtre(i))]);
end
```

Filtre de suavitzat 7x7



Filtre de suavitzat 13x13



3. Quina es la relació entre la mida del filtre de suavitzat que heu utilitzat i el gruix de les línies de contorn obtingudes?

La relació entre la mida del filtre de suavitzat i el gruix de les línies de contorn es pot entendre en termes dels efectes de la suavitat de la imatge i la reducció del soroll.

Quan apliquem un filtre de suavitzat més gran (mida 7x7 o 13x13), es realitza una suavitat més gran a la imatge. Això significa que les variacions de píxel a píxel es difuminen més, reduint el contrast i la definició dels contorns. Com a resultat, les línies de contorn poden aparèixer més amples o més borroses.

Un filtre de suavitzat més gran difuminarà més la imatge, cosa que pot conduir a la detecció de contorns menys precisos o més amplis. Això pot ser útil en algunes aplicacions on es volen reduir els efectes del soroll o les petites variacions de píxel, però també pot conduir a una pèrdua de detall en els contorns.

Exercici 4: Comparació entre mètodes d'obtenció de línies de contorns

L'objectiu de l'exercici és analitzar les diferències entre dos mètodes semblants per obtenir les imatges de contorns (binàries): el filtre Sobel+thresholding i l'algorisme de Canny (utilitzant tots els seus paràmetres!).

A nivell teòric, quines diferències hi ha entre els dos mètodes a nivell d'operacions i cost computacional?

El **filtre Sobel + thresholding** és un mètode senzill i ràpid per detectar contorns en una imatge. Implica l'aplicació de l'operador Sobel per calcular les derivades de primera ordre en direcció horitzontal i vertical. Posteriorment, es calcula el gradient de magnitud combinant aquestes derivades i s'aplica un algorisme de thresholding per convertir el gradient en una imatge binària de contorns. Aquest mètode té un cost computacional relativament baix, ja que consisteix principalment en càlculs de convolució i comparacions píxel a píxel.

Per contra, l'**algorisme de Canny** és un mètode més sofisticat i precís per a la detecció de contorns. Aquest algorisme implica diversos passos, com ara l'aplicació d'un filtre Gaussià per reduir el soroll de la imatge, el càlcul del gradient utilitzant els operadors Sobel, la supressió de no-màxims per subratllar els píxels que formen part dels contorns principals, i la detecció de doble thresholding per determinar quins píxels són realment part dels contorns. Tot i que ofereix resultats més precisos, l'algorisme de Canny té un cost computacional més alt, ja que implica una sèrie de passos i operacions addicionals.

1. *Obteniu els contorns de les imatges contingudes a la carpeta Imatges2 amb els dos mètodes indicats (Sobel+thresholding i Canny). Compareu els resultats obtinguts. Important que pel primer mètode no feu servir la eina «edge» de Matlab. A continuació contesteu a les següents preguntes:*

```
imatge = cdata;

imatge_grisos = im2gray(imatge);

sobel_horitzontal = [-1, 0, 1; -2, 0, 2; -1, 0, 1];
sobel_vertical = [-1, -2, -1; 0, 0, 0; 1, 2, 1];

grad_horitzontal = conv2(imatge_grisos, sobel_horitzontal, 'same');
grad_vertical = conv2(imatge_grisos, sobel_vertical, 'same');
magnitud_gradient = sqrt(grad_horitzontal.^2 + grad_vertical.^2);

llindar_sobel = 100;
contorns_sobel = magnitud_gradient > llindar_sobel;

contorns_canny = edge(imatge_grisos, 'canny');

figure;
subplot(1,3,1);
imshow(imatge_grisos);
title('Imatge Original');

subplot(1,3,2);
imshow(contorns_sobel);
title('Contorns amb Sobel + Thresholding');

subplot(1,3,3);
imshow(contorns_canny);
title('Contorns amb Canny');
```

Imatge Lenna_bn

Imatge Original



Contorns amb Sobel + Thresholding



Contorns amb Canny

*Imatge lena_low*

Imatge Original



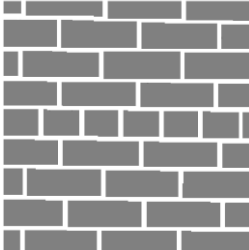
Contorns amb Sobel + Thresholding



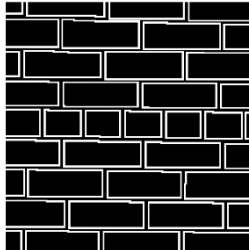
Contorns amb Canny

*Imatge d95a*

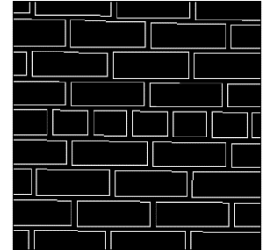
Imatge Original



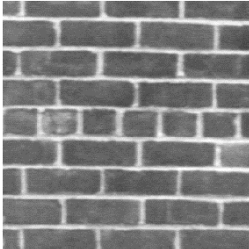
Contorns amb Sobel + Thresholding



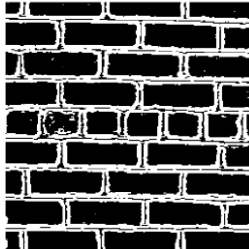
Contorns amb Canny

*Imatge d95*

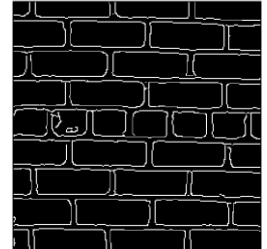
Imatge Original



Contorns amb Sobel + Thresholding



Contorns amb Canny

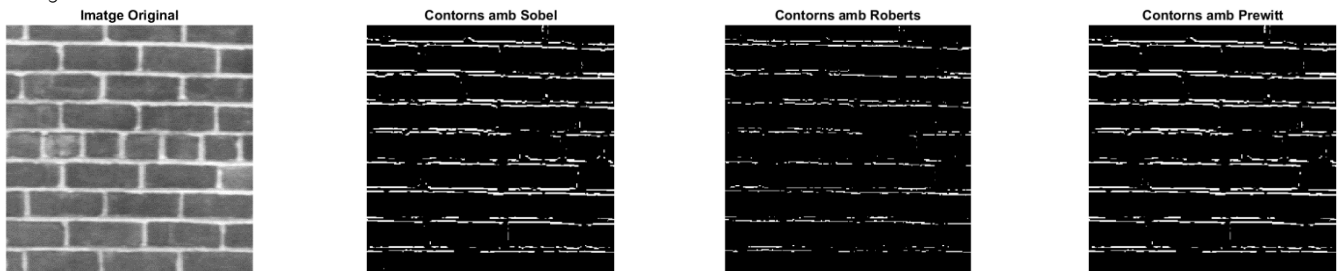
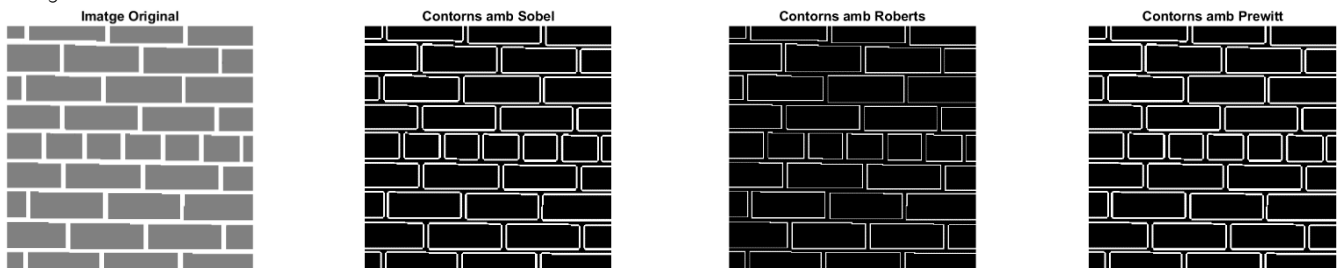


· *Analitzant els resultats obtinguts i tenint en compte el cost del càlcul, quin és el mètode més adequat per a cada imatge?*

En imatges com les d95, el filtre Sobel juntament amb el thresholding ofereix resultats satisfactoris en la detecció de contorns, donant una combinació de simplicitat i eficàcia. No obstant això, en imatges més complexes com les lena, on es necessita capturar tots els detalls fins i tot en regions de baix contrast o amb soroll, l'algoritme de Canny és preferible malgrat el seu cost computacional més alt. Aquest mètode proporciona una detecció de contorns més precisa i robusta, essent la millor opció per a aquest tipus d'imatges que requereixen una alta qualitat de detecció de contorns.

2. Amb les imatges de les toixanes, experimenteu també amb altres opcions de l'eina de Matlab `edge` (ex: 'nothinning') tant pels filtre Sobel com per altres com: Roberts, Prewitt. Com canvien els contorns segons el mètode utilitzat?

```
imatge = d95a;  
  
imatge_grisos = im2gray(imatge);  
  
contorns_sobel = edge(imatge_grisos, 'Sobel', 'nothinning');  
contorns_roberts = edge(imatge_grisos, 'Roberts', 'nothinning');  
contorns_prewitt = edge(imatge_grisos, 'Prewitt', 'nothinning');  
  
figure;  
subplot(2,2,1);  
imshow(imatge_grisos);  
title('Imatge Original');  
  
subplot(2,2,2);  
imshow(contorns_sobel);  
title('Contorns amb Sobel');  
  
subplot(2,2,3);  
imshow(contorns_roberts);  
title('Contorns amb Roberts');  
  
subplot(2,2,4);  
imshow(contorns_prewitt);  
title('Contorns amb Prewitt');
```

Imatge d95a*Imatge d95*

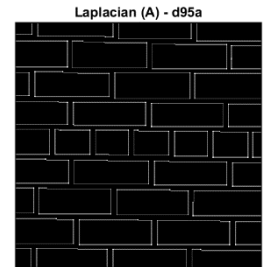
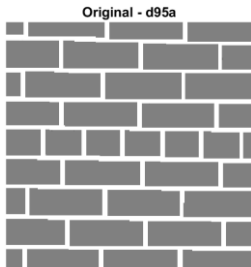
Exercici 5: Detecció de contorns amb la Laplaciana

El filtre Laplacà es basa en el càlcul de la segona derivada i s'utilitza per detectar els contorns. Aquests se situen on la funció Laplaciana passa per 0. És més fi que els anteriors detectors doncs accentua les transicions brusques d'intensitat.

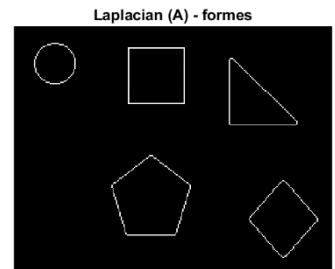
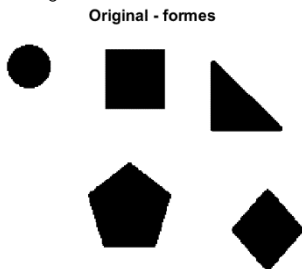
En aquesta ocasió treballareu amb les imatges de la carpeta *Imatges3*.

1. Apliqueu el filtre Laplacà (filtre A) a les imatges sintètiques (*d95a* i *formes*). Per veure el resultat i evitar escalar la imatge resultat feu servir `imshow(ima, [])`. Fer zoom per veure els valors a les vores i mireu directament a la matriu els valors obtinguts.

Imatge *d95a*



Imatge *formes*



```
im = imread('d95a.bmp');
d95a_original_gray = im2gray(im);
im2 = imread('formes.bmp');
formes_original_gray = im2gray(im2);

laplacian_filter_A = [0, 1, 0; 1, -4, 1; 0, 1, 0];

laplacian_d95a_A = conv2(double(d95a_original_gray), laplacian_filter_A, 'same');

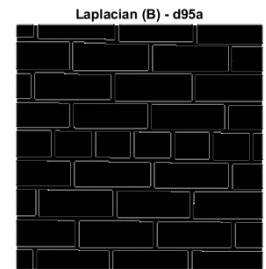
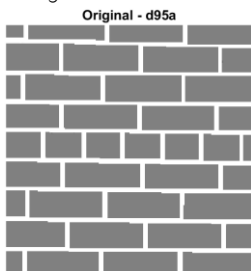
laplacian_formes_A = conv2(double(formes_original_gray), laplacian_filter_A, 'same');

figure;
subplot(2, 3, 1); imshow(d95a_original_gray); title('Original - d95a');
subplot(2, 3, 2); imshow(uint8(laplacian_d95a_A)); title('Laplacian (A) - d95a');

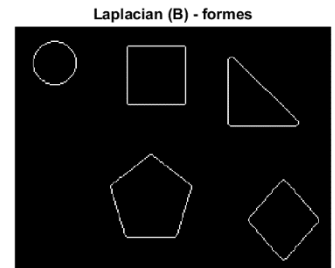
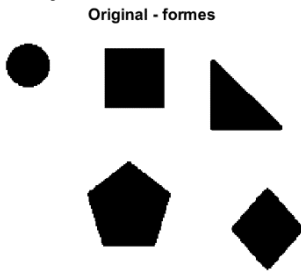
subplot(2, 3, 4); imshow(formes_original_gray); title('Original - formes');
subplot(2, 3, 5); imshow(uint8(laplacian_formes_A)); title('Laplacian (A) - formes');
```

2. Feu el mateix però canviant el signe al filtre (filtre B). El resultat hauria de ser equivalent!

Imatge d95a



Imatge formes



```
im = imread('d95a.bmp');
d95a_original_gray = im2gray(im);
im2 = imread('formes.bmp');
formes_original_gray = im2gray(im2);

laplacian_filter_B = [0, -1, 0; -1, 4, -1; 0, -1, 0];

laplacian_d95a_B = conv2(double(d95a_original_gray), laplacian_filter_B, 'same');

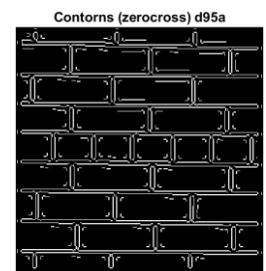
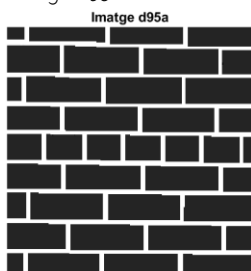
laplacian_formes_B = conv2(double(formes_original_gray), laplacian_filter_B, 'same');

figure;
subplot(2, 3, 1); imshow(d95a_original_gray); title('Original - d95a');
subplot(2, 3, 3); imshow(uint8(laplacian_d95a_B)); title('Laplacian (B) - d95a');

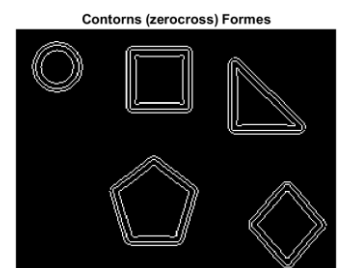
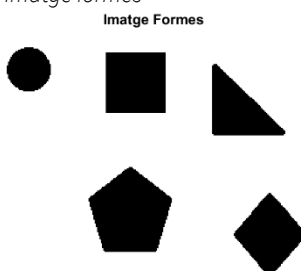
subplot(2, 3, 4); imshow(formes_original_gray); title('Original - formes');
subplot(2, 3, 6); imshow(uint8(laplacian_formes_B)); title('Laplacian (B) - formes');
```

3. Ara calculeu els contorns (pas per zero) amb el paràmetre «zerocross» de la funció edge. Els resultats són satisfactoris?

Imatge d95a



Imatge formes



```

imatge_d95a_grisos = im2double(im);
imatge_formes_grisos = im2double(im2);

filtre_Laplacia_invertit = -fspecial('laplacian', 0.5);

imatge_Laplacia_invertit_d95a = imfilter(imatge_d95a_grisos, filtre_Laplacia_invertit);
imatge_Laplacia_invertit_formes = imfilter(imatge_formes_grisos, filtre_Laplacia_invertit);

contorns_d95a_zerocross = edge(imatge_Laplacia_invertit_d95a, 'zerocross');
contorns_formes_zerocross = edge(imatge_Laplacia_invertit_formes, 'zerocross');

figure;

subplot(2,2,1);
imshow(im, []);
title('Imatge d95a');

subplot(2,2,2);
imshow(contorns_d95a_zerocross);
title('Contorns (zerocross) d95a');

subplot(2,2,3);
imshow(im2, []);
title('Imatge Formes');

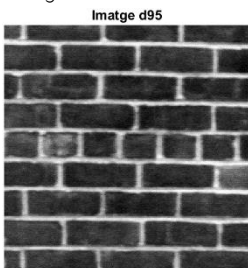
subplot(2,2,4);
imshow(contorns_formes_zerocross);
title('Contorns (zerocross) Formes');

```

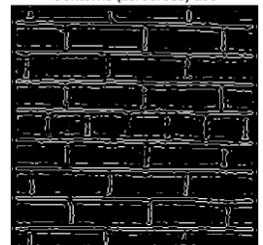
Els resultats obtinguts no són del tot satisfactoris, ja que malgrat que es detecten les formes desitjades, també es generen contorns duplicats o fins i tot triplicats. Aquesta duplicació de contorns pot ser problemàtica a l'hora de treballar amb els resultats obtinguts, ja que pot conduir a una interpretació errònia de la informació i a dificultats en el processament ulterior de les imatges.

4. Repetiu el pas anterior però amb imatges reals (d95 i Lenna_bn). Els contorns obtinguts són els esperats? Quin problema hi ha?

Imatge d95



Contorns (zerocross) d95



Imatge Lenna_bn



Contorns (zerocross) Lenna_n



```

imatge_d95 = cdata;
imatge_Lenna_bn = Lenna_bn;

imatge_d95_grisos = im2double(imatge_d95);
imatge_Lenna_bn_grisos = im2double(imatge_Lenna_bn);

filtre_Laplacia_invertit = -fspecial('laplacian', 0.5);

imatge_Laplacia_invertit_d95 = imfilter(imatge_d95_grisos, filtre_Laplacia_invertit);
imatge_Laplacia_invertit_Lenna_bn = imfilter(imatge_Lenna_bn_grisos, filtre_Laplacia_invertit);

contorns_d95_zerocross = edge(imatge_Laplacia_invertit_d95, 'zerocross');
contorns_Lenna_bn_zerocross = edge(imatge_Laplacia_invertit_Lenna_bn, 'zerocross');

figure;

subplot(2,2,1);
imshow(imatge_d95_grisos, []);
title('Imatge d95');

subplot(2,2,2);
imshow(contorns_d95_zerocross);
title('Contorns (zerocross) d95');

subplot(2,2,3);
imshow(imatge_Lenna_bn_grisos, []);
title('Imatge Lenna_bn');

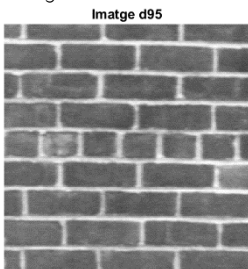
subplot(2,2,4);
imshow(contorns_Lenna_bn_zerocross);
title('Contorns (zerocross) Lenna_bn');

```

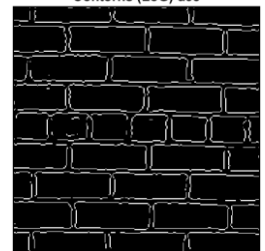
Els resultats que s'han obtingut mostren una manca de precisió notable, ja que es poden observar contorns dobles en diverses parts de les dues imatges analitzades. Aquesta manca de precisió pot dificultar la interpretació correcta de les formes presents a les imatges i limitar la seva utilitat per a aplicacions posteriors.

5. La solució seria «suavitzar» primer la imatge, per això hi ha el filtre Laplacian of Gaussian (LoG). Apliqueu-lo amb diferents valors de sigma (ex: 0,5).

Imatge d95

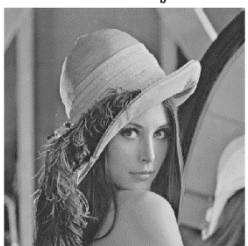


Contorns (LoG) d95



Imatge Lenna_bn

Imatge Lenna_bn



Contorns (LoG) Lenna_bn



Emprant valors de sigma superiors a 0,5, hem obtingut resultats considerats bastant bons per a la primera imatge, i fins i tot millors que els anteriors per a la segona imatge. Cal destacar que l'exemple anterior s'ha realitzat amb un valor de sigma igual a 1.

```
imatge_d95_grisos = im2double(imatge_d95);
imatge_Lenna_bn_grisos = im2double(imatge_Lenna_bn);

sigma = 1;

imatge_Log_d95 = imfilter(imatge_d95_grisos, fspecial('gaussian', [5 5], sigma)),
fspecial('laplacian');
imatge_Log_Lenna_bn = imfilter(imatge_Lenna_bn_grisos, fspecial('gaussian', [5 5], sigma)),
fspecial('laplacian');

contorns_Log_d95 = edge(imatge_Log_d95, 'zerocross');
contorns_Log_Lenna_bn = edge(imatge_Log_Lenna_bn, 'zerocross');

figure;

subplot(2,2,1);
imshow(imatge_d95_grisos);
title('Imatge d95');

subplot(2,2,2);
imshow(contorns_Log_d95);
title('Contorns (LoG) d95');

subplot(2,2,3);
imshow(imatge_Lenna_bn_grisos);
title('Imatge Lenna_bn');

subplot(2,2,4);
imshow(contorns_Log_Lenna_bn);
title('Contorns (LoG) Lenna_bn');
```

Exercici 6: Conclusions sobre els detectors de contorns

De les diferents tècniques amb les que has treballat als anteriors exercicis per trobar els contorns (Sobel, Sobel+thresholding, Canny, Laplacà, LoG), quin/quins triaries per obtenir els contorns de les imatges «formes» i «d95»?

Per fer la valoració tingúes en compte també el cost computacional. Raona la teva resposta i indica les instruccions que has utilitzat.

En el cas de la imatge "formes", els resultats més òptims es van obtenir utilitzant el filtre Laplacian, mentre que en la imatge "d95", els millors resultats es van assolir amb l'ús de LoG (Laplacian of Gaussian), després de suavitzar la imatge.

Exercici 7: Ús dels contorns per millorar la imatge (Edge Enhancement, Sharpening)

Bàsicament hi ha dos estratègies per guanyar definició (emfatitzar els contorns):

- $Imatge_resultant = Imatge + Laplaciana(Imatge)$
- $Imatge_resultant = Imatge - Imatge_suavitzada$ (Unsharp masking)

Sobre la imatge «Lenna_bn_s2» (carpeta Imatges3):

1. Apliqueu la primera estratègia per millorar la imatge. Feu servir el filtre Laplaciana (B) i d'altres variants que indico a sota.

```
im = imread('Lenna_bn-s2.png');

laplacian_0 = [0 -1 0; -1 4 -1; 0 -1 0];
laplacian_1 = [-1, -1, -1; -1, 9, -1; -1, -1, -1];
laplacian_2 = [1, -2, 1; -2, 5, -2; 1, -2, 1];
laplacian_3 = [0, -1, 0; -1, 5, -1; 0, -1, 0];

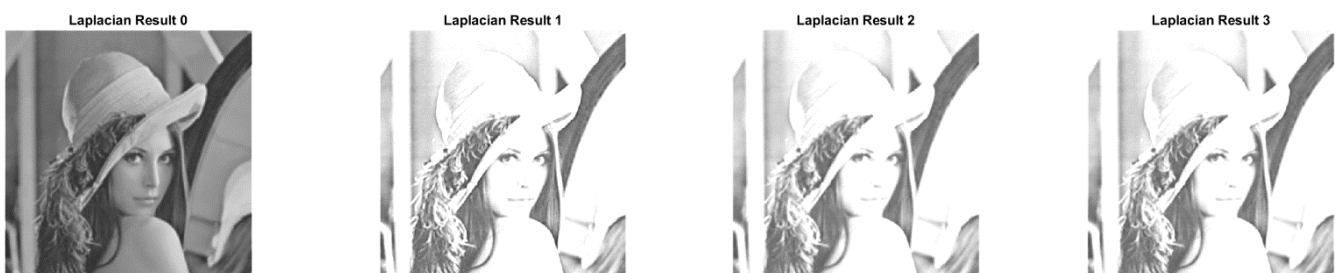
laplacian_result_0 = im + uint8(imfilter(double(im), laplacian_0, "conv"));
laplacian_result_1 = im + uint8(imfilter(double(im), laplacian_1, "conv"));
laplacian_result_2 = im + uint8(imfilter(double(im), laplacian_2, "conv"));
laplacian_result_3 = im + uint8(imfilter(double(im), laplacian_3, "conv"));

figure;
subplot(2, 2, 1);
imshow(laplacian_result_0);
title('Laplacian Result 0');

subplot(2, 2, 2);
imshow(laplacian_result_1);
title('Laplacian Result 1');

subplot(2, 2, 3);
imshow(laplacian_result_2);
title('Laplacian Result 2');

subplot(2, 2, 4);
imshow(laplacian_result_3);
title('Laplacian Result 3');
```

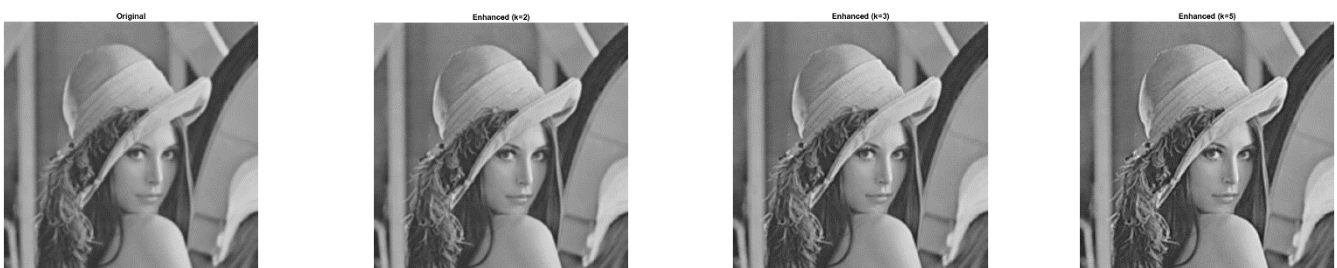


2. Aplica la segona estratègia però amb la següent fórmula genèrica:

$$Imatge_resultant = Imatge + k * gmask$$

essent $gmask = Imatge - Imatge_suavitzada$, k es una constant ($k \geq 0$)

Feu servir un filtre de suavitzat petit (3x3).



```
img = Lenna_bn_s2;
img_gray = im2gray(img);

smoothed_img = imgaussfilt(img_gray, 1);

gmask = double(img_gray) - double(smoothed_img);

k = 5;

enhanced_img = uint8(double(img_gray) + k * gmask);

figure;
subplot(1,2,1); imshow(img_gray); title('Original');
subplot(1,2,2); imshow(enhanced_img); title(['Enhanced (k=', num2str(k), ')']);
```